MONGODB — EXAM NOTES

Definitions + Syntax + Examples + Important Exam Differences

1. What is MongoDB?

Definition: MongoDB is a NoSQL, document-oriented database management system that stores data in the form of documents instead of traditional rows and tables.

MongoDB stores documents in BSON (Binary JSON) format.

Important terms:

MongoDB	Relational Database
Database	Database
Collection	Table
Document	Row
Field	Column

Example document:

```
{
    name: "Harry",
    lang: "JavaScript",
    member_since: 5
}
```

2. Database Commands

2.1 Show all databases

Definition: The show dbs command displays all available databases.

```
show dbs
```

2.2 Create or switch to a database

Definition: The use command is used to switch to an existing database or select a database context.

```
use dbName
use college
```

Note: The database is actually created when data is stored in it.

2.3 View current database

Definition: The db command displays the name of the currently selected database.

```
db
```

2.4 Delete current database

Definition: dropDatabase() deletes the currently selected database and its data.

```
db.dropDatabase()
use college
db.dropDatabase()
```

3. Collection Commands

Definition: A collection is a group of MongoDB documents. It is similar to a table in a relational database.

3.1 Show collections

```
show collections
```

3.2 Create a collection

Definition: createCollection() explicitly creates a collection.

```
db.createCollection("comments")
```

3.3 Drop a collection

Definition: drop() deletes a collection and all documents inside it.

```
db.comments.drop()
```

4. Document Commands — CRUD

CRUD stands for Create, Read, Update and Delete.

4.1 Insert One Document

Definition: insertOne() inserts a single document into a collection.

```
db.comments.insertOne({
    name: "Harry",
    lang: "JavaScript",
    member_since: 5
})
```

4.2 Insert Many Documents

Definition: insertMany() inserts multiple documents at once.

```
db.comments.insertMany([
    {name: "Harry", lang: "JavaScript", member_since: 5},
    {name: "Rohan", lang: "Python", member_since: 3},
    {name: "Lovish", lang: "Java", member_since: 4}
])
```

- **insertOne() → one document**
- **insertMany() → multiple documents**

4.3 Read / Find Documents

Definition: find() retrieves documents from a collection.

```
db.comments.find()
```

Pretty format:

```
db.comments.find().pretty()
```

Definition: findOne() returns the first document matching a specified condition.

```
db.comments.findOne({name: "Harry"})
```

4.4 Search by Field

Definition: find() can be used with a condition to search documents based on a particular field.

```
db.comments.find({lang: "Python"})
```

4.5 Limit

Definition: limit() restricts the number of documents returned by a query.

```
db.comments.find().limit(2)
```

4.6 Skip

Definition: skip() skips a specified number of documents.

```
db.comments.find().skip(2)
db.comments.find().skip(2).limit(2)
```

Skip + limit is useful for pagination.

4.7 Count Documents

Definition: countDocuments() counts the number of documents matching a condition.

```
db.comments.countDocuments({lang: "Python"})
```

5. Update Commands

5.1 updateOne()

Definition: updateOne() updates the first document that matches a specified condition.

```
db.comments.updateOne(
    {name: "Shubham"},
    {$set: {name: "Harry", lang: "JavaScript", member_since: 51}}
)
```

5.2 $set

Definition: $set sets or modifies the value of a field.

```
db.comments.updateOne(
    {name: "Rohan"},
    {$set: {lang: "Python"}}
)
```

5.3 Upsert

Definition: Upsert means update if the document exists; otherwise, insert a new document.

```
db.comments.updateOne(
    {name: "Shubham"},
    {$set: {lang: "JavaScript"}},
    {upsert: true}
)
```

5.4 updateMany()

Definition: updateMany() updates all documents matching a condition.

```
db.comments.updateMany(
    {lang: "JavaScript"},
    {$set: {verified: true}}
)
```

5.5 Increment a Field — $inc

Definition: $inc increases or decreases a numeric field by a specified amount.

```
db.comments.updateOne(
    {name: "Rohan"},
    {$inc: {member_since: 2}}
)
```

5.6 Rename a Field — $rename

Definition: $rename changes the name of an existing field.

```
db.comments.updateOne(
    {name: "Rohan"},
    {$rename: {member_since: "member"}}
)
```

6. Delete Commands

6.1 deleteOne()

Definition: deleteOne() deletes the first document matching a specified condition.

```
db.comments.deleteOne({name: "Harry"})
```

6.2 deleteMany()

Definition: deleteMany() deletes all documents matching a specified condition.

```
db.comments.deleteMany({lang: "Java"})
```

7. Query Operators

Query operators are used to specify conditions in MongoDB queries.

7.1 Comparison Operators

Operator	Meaning
$lt	Less than
$lte	Less than or equal
$gt	Greater than
$gte	Greater than or equal
$ne	Not equal

```
db.comments.find({member_since: {$lt: 90}})
db.comments.find({member_since: {$lte: 90}})
db.comments.find({member_since: {$gt: 90}})
db.comments.find({member_since: {$gte: 90}})
db.comments.find({member_since: {$ne: 5}})
```

7.2 Logical Operators

$and

Definition: $and returns documents where all specified conditions are true.

```
db.comments.find({$and: [{lang: "Python"}, {member_since: {$gt: 2}}]})
```

$or

Definition: $or returns documents where at least one specified condition is true.

```
db.comments.find({$or: [{lang: "Python"}, {lang: "Java"}]})
```

$in

Definition: $in selects documents where a field value matches any value in a specified list.

```
db.comments.find({lang: {$in: ["Python", "Java"]}})
```

$nin

Definition: $nin selects documents where a field value does not match any value in a specified list.

```
db.comments.find({lang: {$nin: ["C++", "Rust"]}})
```

7.3 Sorting

Definition: sort() arranges query results in ascending or descending order.

```
db.comments.find().sort({member_since: 1})
db.comments.find().sort({member_since: -1})
```

- **1 → Ascending**
- **-1 → Descending**

8. Projection

Definition: Projection selects only specific fields from the documents returned by a query.

```
db.comments.find({}, {name: 1, lang: 1, _id: 0})
```

- **1 → include field**
- **0 → exclude field**

9. Indexes — Basics

Definition: An index is a data structure used by MongoDB to improve the speed of queries on specified fields.

Think of an index like the index of a book.

9.1 Create an index

```
db.comments.createIndex({name: 1})
```

9.2 View indexes

```
db.comments.getIndexes()
```

9.3 Drop an index

```
db.comments.dropIndex({name: 1})
```

10. Aggregation — Basics

Definition: Aggregation processes documents and performs calculations or transformations on groups of documents.

10.1 Count by Field

```
db.comments.aggregate([
    {$group: {_id: "$lang", total: {$sum: 1}}}
])
```

$group groups documents and $sum performs the count/sum operation.

10.2 Average by Field

```
db.comments.aggregate([
    {$group: {_id: "$lang", avgExperience: {$avg: "$member_since"}}}
])
```

This groups documents by lang and calculates the average member_since for each language.

11. Important Exam Differences

insertOne() vs insertMany()

Command	Meaning
insertOne	Inserts one document
insertMany	Inserts multiple documents

updateOne() vs updateMany()

Command	Meaning
updateOne	Updates the first matching document
updateMany	Updates all matching documents

deleteOne() vs deleteMany()

Command	Meaning
deleteOne	Deletes the first matching document
deleteMany	Deletes all matching documents

12. Complete MongoDB Example for Exam

```
use college

db.createCollection("students")

db.students.insertOne({
    name: "Zainab",
    age: 19,
    course: "AI"
})

db.students.insertMany([
    {name: "Zainab", age: 19, course: "AI"},
    {name: "Sara", age: 20, course: "CS"},
    {name: "Ayesha", age: 19, course: "AI"}
])

db.students.find()

db.students.find({course: "AI"})

db.students.updateOne(
```

```
    {name: "Zainab"},
    {$set: {age: 20}}
)

db.students.deleteOne({name: "Sara"})

db.students.find().sort({age: 1})

db.students.find({}, {name: 1, course: 1, _id: 0})
```

13. Last-Minute Revision Sheet

```
show dbs
use college
db
db.dropDatabase()

show collections
db.createCollection("comments")
db.comments.drop()

db.comments.insertOne({...})
db.comments.insertMany([...])

db.comments.find()
db.comments.find().pretty()
db.comments.findOne({...})
db.comments.find({...})

db.comments.find().limit(2)
db.comments.find().skip(2)
db.comments.countDocuments({...})

db.comments.updateOne({...}, {$set: {...}})
db.comments.updateMany({...}, {$set: {...}})
db.comments.updateOne({...}, {$inc: {...}})
db.comments.updateOne({...}, {$rename: {...}})

db.comments.deleteOne({...})
db.comments.deleteMany({...})

$lt   → <
$lte  → ≤
$gt   → >
$gte  → ≥
$ne   → ≠
$and  → AND
$or   → OR
$in   → matches any listed value
$nin  → does not match listed values

db.comments.find().sort({member_since: 1})
db.comments.find().sort({member_since: -1})

db.comments.find({}, {name: 1, lang: 1, _id: 0})

db.comments.createIndex({name: 1})
```

```
db.comments.getIndexes()
db.comments.dropIndex({name: 1})

db.comments.aggregate([
    {$group: {_id: "$lang", total: {$sum: 1}}}
])
```

Based on the uploaded MongoDB Cheatsheet (7 pages).